

CCNA 200-301 V2.0

03 Ethernet, Frames and Switching

Part I — Foundations

EXAM TOPICS

1.1

LECTURER

Dr. Mustafa Hameed

THE ISLAMIA UNIVERSITY OF BAHAWALPUR

Department of Information Technology

What you will be able to do

- **Describe** the Ethernet frame and the purpose of each field you will actually meet on a device.
- **Explain** how a switch learns, forwards, floods and ages MAC addresses — the algorithm behind every switching chapter that follows.
- **Read** the MAC address table and use it to prove where a device is physically connected.
- **Predict** what a switch does with a unicast, broadcast or unknown destination.

Before we start

Chapter 1 for frames and broadcast domains; Chapter 2 for the link underneath.

Vocabulary for this chapter

Ethernet

MAC address

OUI

unicast

broadcast

multicast

CAM table

MAC address table

learning

flooding

filtering

ageing

unknown unicast

MTU

FCS

Each is defined where it first matters — not here.

Hexadecimal, and why it exists

Ordinary decimal counts in tens using ten symbols, 0 to 9. **Hex** counts in sixteens using sixteen symbols: 0 to 9, then **a b c d e f** for ten to fifteen.

It exists because **one hex digit is exactly four bits**, so two hex digits are exactly one byte. Writing a 48-bit MAC address in decimal would take fifteen digits with no relationship to the underlying bits; in hex it takes twelve, and each one maps to a nibble you can reason about.

The three things you actually have to do with it

- **Recognise it.** A leading `0x` means “what follows is hex” — `0x0800` is the number 2048, not eight hundred. Cisco also writes hex with no prefix at all, as in `00d0.58a1.0201`.
- **Expand one digit to four bits**, using the table above. This is all that EUI-64 (Chapter 5) and the MAC address bits below require.
- **Know that `f` is all ones.** So `ffff.ffff.ffff` is forty-eight 1 bits — every bit set — which is exactly why that is the broadcast address.

You will never be asked to convert a long hex number to decimal in your head. You will regularly be asked what one digit means in bits.

What is the first byte of 00d0.58a1.0201 in bits?

The first byte is the first *two* hex digits, 00.

From the table, 0 is 0000. So 00 is 0000 0000 — eight zero bits.

That matters immediately: the conceptbox after the frame explains that the low-order bit of the first byte says unicast or multicast, and the next bit up says whether the address was assigned by a manufacturer. Both are zero here, so this is a unicast address burned in by its vendor. In Chapter 28 you will meet an address where the second bit is *one*, and that single bit is the whole diagnosis.

The Ethernet frame, field by field

- **Preamble** — clock synchronisation. You will never see it.
- **Destination MAC** (6 bytes) — who this frame is for. The switch reads this to decide where to send it.
- **Source MAC** (6 bytes) — who sent it. The switch reads this to *learn*.
- **Type/Length** (2 bytes) — what is inside. `0x0800` is IPv4, `0x86DD` is IPv6, `0x8100` means a VLAN tag follows (Chapter 10).
- **Payload** — 46 to 1500 bytes. Below 46 the frame is padded; this is why a frame under 64 bytes total is a **runt**.

The Ethernet frame, field by field (continued)

- **FCS** (4 bytes) — a checksum. If it fails, the frame is counted as a CRC error and discarded (Chapter 2).

Three kinds of destination

- **Unicast** — one specific device. The low-order bit of the first byte is 0.
- **Broadcast** — `ffff.ffff.ffff`, every device in the VLAN. This is what defines a broadcast domain.
- **Multicast** — a group. The low-order bit of the first byte is 1. OSPF (Chapter 17), HSRP and VRRP (Chapter 19) all use multicast.

Learn, then forward or flood

For every frame that arrives on a port:

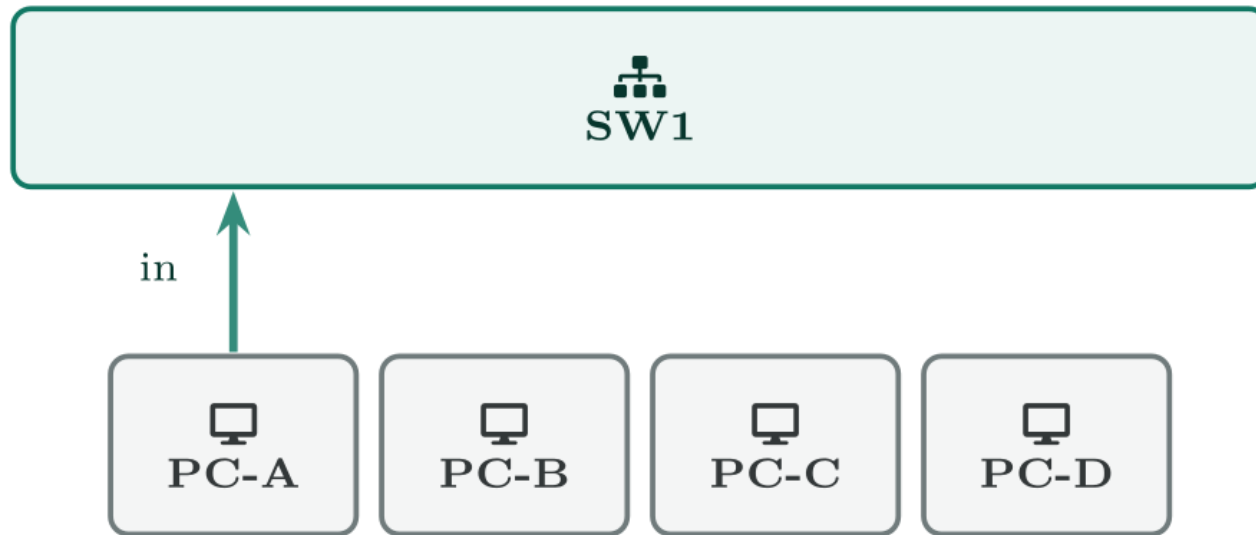
- **Learn.** Record the *source* MAC address against the port it arrived on, in this VLAN. Reset the ageing timer if it is already there.
- **Decide on the destination.**
- Known unicast → **forward** out that one port.
- Unknown unicast → **flood** out every other port in the VLAN.
- Broadcast or multicast → **flood**.

Learn, then forward or flood (continued)

- Destination is on the same port it arrived on → **filter**, that is, discard it.

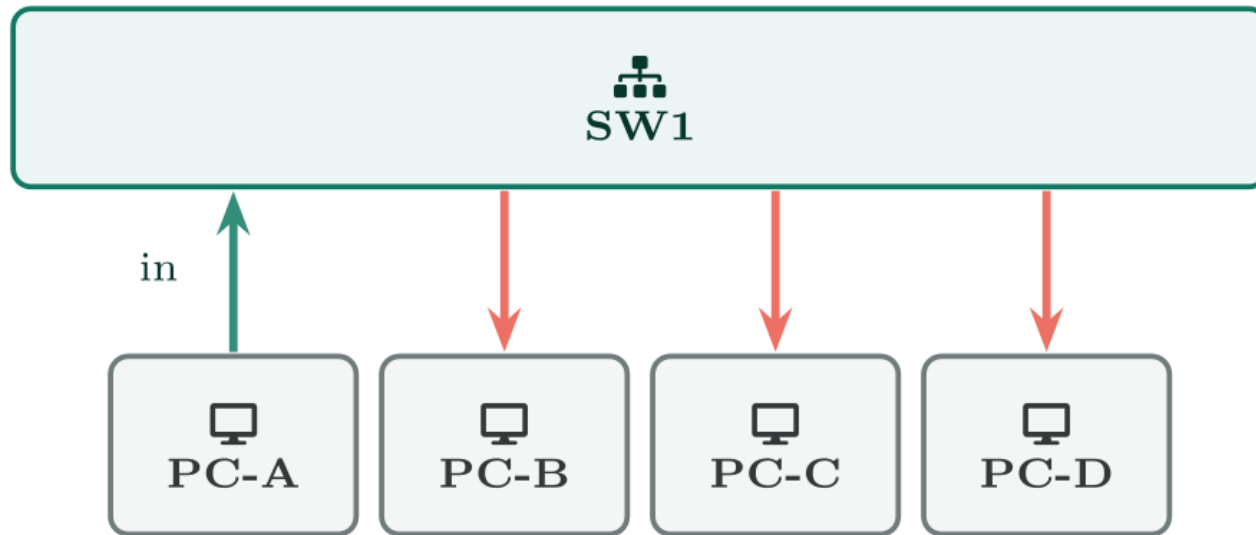
A switch never modifies the addresses in a frame it forwards, and it never consults an IP address. Both of those are the router's job.

How a switch actually works



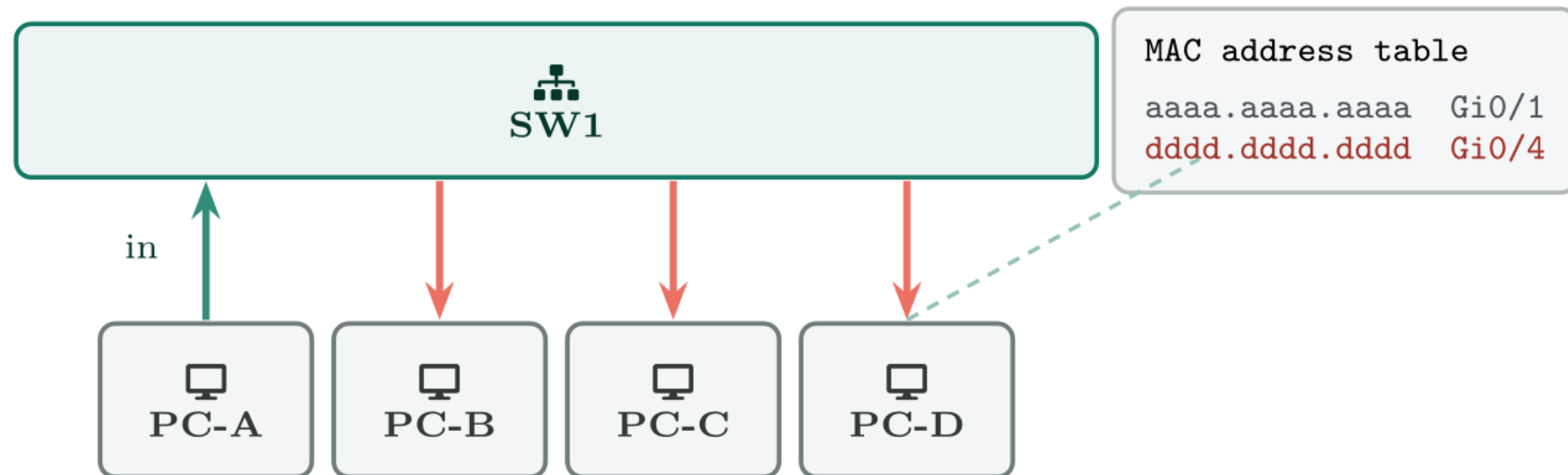
Destination not in the table. Now what?

How a switch actually works



Flood: every port in the VLAN except the one it arrived on

How a switch actually works



PC-D replies — and the switch has now learned where it is

The whole switching algorithm. Flooding is not a failure — it is how the table gets built, and the second frame between the same two hosts is already efficient.

The consequence that matters: unknown unicast flooding

A switch floods a unicast frame when it does not know the destination. That is not a fault — it is how the network keeps working while the table is learned. But it means a full or wiped MAC table causes traffic to be sprayed to every port, which is both a performance problem and a security one. It is also the mechanism a MAC flooding attack abuses, which is why Chapter 25 exists and why topic **4.7** names port security.

show mac address-table

```
SW1# show mac address-table
```

Mac Address Table

```
-----  
Vlan    Mac Address      Type      Ports  
----    -  
1       00d0.58a1.0201   DYNAMIC   Gi0/1  
1       0060.5c2b.19a4   DYNAMIC   Gi0/2  
10      000a.f36e.7712   DYNAMIC   Gi0/5  
10      0002.1751.4c33   STATIC    Gi0/6  
20      00e0.b05f.2210   DYNAMIC   Gi0/8  
Total Mac Addresses for this criterion: 5
```

A static mac entry, and a shorter ageing time

Configure

```
! Pin a known device to a known port. Rarely needed, but it survives  
! ageing and cannot be moved by a spoofed frame.  
mac address-table static 0002.1751.4c33 vlan 10 interface GigabitEthernet0/6  
!  
! Age entries out faster than the 300-second default.  
mac address-table aging-time 120
```

Following a frame across two switches

PC-A (00d0.58a1.0201) on SW1 Gi0/1 sends to PC-B (00e0.b05f.2210) on SW2 Gi0/3. Both switches have empty tables.

- The frame arrives at SW1 Gi0/1. SW1 *learns* 00d0.58a1.0201 on Gi0/1.
- SW1 does not know PC-B, so it *floods* — out its uplink and every other access port.
- SW2 receives it on its uplink and learns 00d0.58a1.0201 on *that uplink port*, not on Gi0/1. A switch only ever knows the direction, not the distance.
- SW2 does not know PC-B either, so it floods. PC-B receives it.
- PC-B replies. Now SW2 learns PC-B on Gi0/3, and because it already knows PC-A, it *forwards* rather than flooding.

Following a frame across two switches (continued)

- SW1 learns PC-B on its uplink. From here on the conversation is unicast in both directions.

Two floods to start a conversation, then none. That is the whole design.

How a switch actually works

- **Expecting a switch to know a distance.** The table says which port, never how many hops. Five hundred devices can appear behind one uplink port and that is entirely normal.
- **Reading a stale table.** An entry can be up to five minutes old. If you have just moved a cable, the table may still show the old port.
- **Confusing the MAC table with the ARP table.** The MAC table maps MAC → port and lives on a switch. The ARP table maps IP → MAC and lives on any device with an IP address. The exam will test that you know which to look at.
- **Forgetting the VLAN column.** The same MAC address can legitimately appear in two VLANs. Learning is always per VLAN.

Two ports, one MAC address, and a flapping log.

Users on VLAN 10 report intermittent drops. The log shows:

Two ports, one MAC address, and a flapping log.

What would you check first — and what do you expect to see?

show logging | include MACFLAP

```
SW1# show logging | include MACFLAP
```

```
%SW_MATM-4-MACFLAP_NOTIF: Host 000a.f36e.7712 in vlan 10 is flapping  
  between port Gi0/5 and port Gi0/12
```


What is the fault — and what does this output rule out?

Why it is doing that

Diagnosis. The switch is learning the same source MAC on two ports in quick succession. A device cannot be in two places, so one of three things is true: there is a **physical loop** joining those ports (the same frame is arriving by two paths — see Chapter 12), a device is **spoofing** that address, or two devices have been given the **same MAC** by a bad clone or a virtualisation misconfiguration.

And the lesson

Fix. Check spanning tree first, because a loop is both the most likely cause and the most damaging: `show spanning-tree vlan 10` and look for a port that should be blocking and is not. If the topology is loop-free, shut `Gi0/12`, confirm the flapping stops, and trace what is attached to it.

Watching a switch learn

Topology. Two switches, uplinked; two PCs on **SW1**, one on **SW2**. All in VLAN 1 for now.

1. Power everything on and immediately run `show mac address-table` on both switches. Record what is there before any traffic.
2. Ping from PC-A to PC-C. Re-run the command on both switches and record which addresses each has learned, and against which port.
3. Explain, from your own capture, why **SW2** shows PC-A against its uplink rather than against a host port.
4. Run `clear mac address-table dynamic`, then ping again and watch the table rebuild.
5. Set `mac address-table aging-time 15`. Leave a PC idle for thirty seconds and confirm its entry disappears.

Watching a switch learn (continued)

1. Extension. Add a second cable between the two switches to create a loop. Observe what happens to the MAC table and the log before spanning tree settles — then read Chapter 12 to find out why the network survived.

CHECKPOINT 1 of 3

A switch receives a frame whose destination MAC is already in its table, against the same port the frame arrived on. What does it do, and what is that behaviour called?

Discuss · then advance

Checkpoint 1 of 3

A switch receives a frame whose destination MAC is already in its table, against the same port the frame arrived on. What does it do, and what is that behaviour called?

It drops it. The behaviour is called **filtering**: the destination is known to be reachable through the same port the frame arrived on, so forwarding it would send it back where it came from.

CHECKPOINT 2 of 3

Why does the first ping between two devices often show a longer round trip than the ones after it?

Discuss · then advance

Checkpoint 2 of 3

Why does the first ping between two devices often show a longer round trip than the ones after it?

Because the first one triggers an ARP request and possibly a MAC flood, and the round trip includes that resolution. Later pings use the cached entries.

CHECKPOINT 3 of 3

You need to prove which wall port a particular laptop is using.
Which table do you consult, on which device, and what do you need to know first?

Discuss · then advance

Checkpoint 3 of 3

You need to prove which wall port a particular laptop is using. Which table do you consult, on which device, and what do you need to know first?

The MAC address table, on the switch the laptop is connected to. You need the laptop's MAC address first — and note that on wireless it may be randomised (Chapter 28), so the MAC on the label may not be the one on the network.

What to take away

- A switch learns from the *source* address and decides on the *destination* address. Everything else follows.
- Known unicast is forwarded, unknown unicast and broadcast are flooded, and a frame is never sent back out of the port it came in on.
- The MAC address table maps MAC to port, per VLAN, and ages entries out after five minutes by default.
- MAC table is not ARP table. One lives on a switch and maps to ports; the other maps IP to MAC and lives on anything with an IP address.

What to take away (continued)

- A MAC address flapping between two ports means a loop, a spoof, or a duplicate — and you check for the loop first.

Where we got to

- A switch learns from the *source* address and decides on the *destination* address. Everything else follows.
- Known unicast is forwarded, unknown unicast and broadcast are flooded, and a frame is never sent back out of the port it came in on.
- The MAC address table maps MAC to port, per VLAN, and ages entries out after five minutes by default.
- MAC table is not ARP table. One lives on a switch and maps to ports; the other maps IP to MAC and lives on anything with an IP address.

NEXT

Chapter 4 — IPv4 Addressing and Subnetting